

Local MQTT Telemetry & Bidirectional Motor Control

*Single-ESP32 BME280 Sensing and Motor Control over a Self-Hosted
Mosquitto Broker*



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report documents Task 2.7: a single ESP32 board that reads live BME280 environmental data over I2C and exposes full bidirectional control of a DC motor, all mediated through a Mosquitto MQTT broker hosted locally on the developer's own machine rather than a public cloud broker. The ESP32 continuously publishes temperature, humidity, and pressure readings, and continuously publishes its motor's true running state, direction, and speed. In the other direction, it subscribes to three command topics – state, direction, and speed – and applies each incoming command to the motor after validating and, where necessary, clamping it.

This project is a direct architectural descendant of an earlier two-board interlock project (Task 2.4/2.6): the WiFi/broker reconnection pattern was carried over essentially unchanged, and the `MqttInterlockClient` class name was deliberately kept even though its role changed completely, from a narrow interlock-specific transport to a fully generic, topic-agnostic publish/subscribe wrapper. Everything specific to the old cross-board interlock concept – the partner-link timeout, the shared single-byte obstacle payload, the `MY_ID`/`PARTNER_ID` identity pair – was deliberately discarded, since this task has only one board and no partner to synchronize with.

The most significant engineering finding of this project was not in the firmware at all, but in the network layer: Windows tracks a network's Public/Private security profile independently per SSID, so a firewall rule that works on one WiFi network silently stops working the moment the PC joins a different network, even if it is the “same” physical LAN from the ESP32's point of view. A second, unrelated hardware issue – the WiFi extender used for this project performing AP-level client isolation – was found and isolated using the same broker-side logging that ultimately proved the fix worked.

2. System Overview & Goal

Unlike the earlier two-board interlock task, this system has exactly one ESP32, one BME280 sensor, one motor driver, and one broker, all on the same local network:

```
(BME280, I2C) --> [ESP32] --publish--> [Mosquitto 192.168.1.33:1883] --subscribe--> [MQTT Explorer /
                                     mosquitto_sub]
                                     MQTT Explorer / mosquitto_pub
--publish--> [Mosquitto 192.168.1.33:1883] --subscribe--> [ESP32] --drives--> (L298N + DC Motor)
```

- The ESP32 is simultaneously a publisher (sensor + motor status) and a subscriber (motor commands) on the same broker connection.
- There is no peer board and no cross-gating logic: the motor is driven directly by whatever command was last received on the three command topics, subject only to the local disconnect fail-safe (Section).
- The broker itself is not a cloud service – it is Mosquitto 2.1.2 run from a terminal on the developer's own PC, on the same LAN as the ESP32.

3. Hardware Architecture & Pin Mapping

Pin assignments come directly from `config.h`, so there is a single place to retune the wiring without touching application logic elsewhere.

Peripheral	Signal	ESP32 GPIO	Role
BME280 (I2C)	SDA	GPIO 21 (default <code>Wire</code> pin)	I2C data line, shared bus
BME280 (I2C)	SCL	GPIO 22 (default <code>Wire</code> pin)	I2C clock line, shared bus
Motor driver (L298N)	IN1	GPIO 26 (<code>MOTOR_PIN_IN1</code>)	Direction line A
Motor driver (L298N)	IN2	GPIO 27 (<code>MOTOR_PIN_IN2</code>)	Direction line B
Motor driver (L298N)	ENA	GPIO 25 (<code>MOTOR_PIN_PWM</code>)	PWM speed line

Table 1: ESP32 pin mapping for the single-board Task 2.7 build.

`Wire.begin()` is called once, centrally, in `main.cpp`'s `setup()` – deliberately *not* inside `BME280Sensor::begin()` – since ownership of the shared I2C bus belongs to whichever code coordinates all I2C peripherals, not to any single sensor class.

Network prerequisites

The ESP32 and the broker-hosting PC must be reachable to each other at the IP layer, on the same LAN. Section documents in detail why this requirement is not as trivial as it sounds on this particular network.

4. Schematic and Circuit

Wiring Schematic

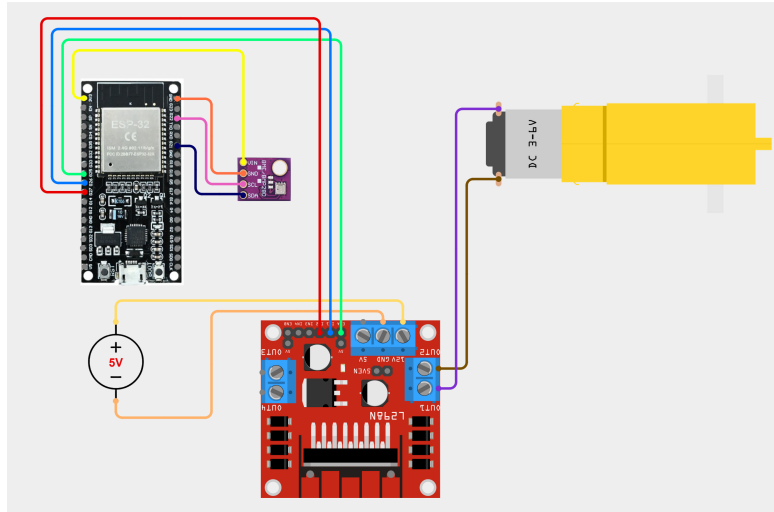


Figure 1: wiring schematic

Assembled Circuit

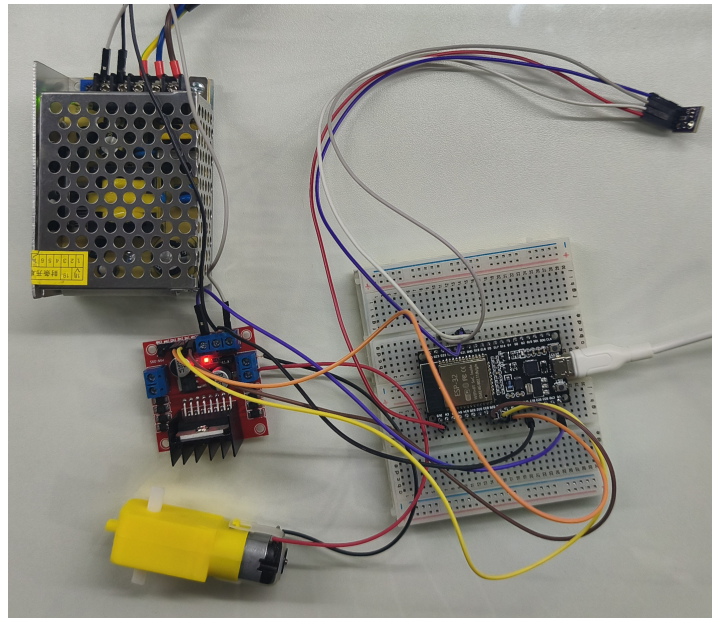


Figure 2: Photo of the assembled circuit

5. Software Architecture

The firmware follows the same single-responsibility split used in the prior interlock project, minus everything interlock-specific, plus one new class for the motor:

File	Responsibility
<code>config.h</code>	Every credential, GPIO pin, topic constant, and timing constant used by the sketch. No other file contains a raw pin number, topic string, or broker address.
<code>BME280Sensor.h/.cpp</code>	Owens the Adafruit BME280 driver instance; exposes cached temperature/humidity/pressure with a combined validity check.
<code>MotorController.h/.cpp</code>	New for this task. Owns direction (IN1/IN2) and PWM speed (ENA via <code>ledc</code>) for the L298N driver; the single source of truth for the motor's actual live state.
<code>MqttInterlockClient.h/.cpp</code>	Reworked from the interlock version into a fully generic, topic-agnostic pub/sub wrapper. Owns WiFi/broker connection and reconnection, generic <code>publish()</code> , and a slot-tagged <code>subscribe()</code> that routes incoming messages without interpreting their meaning.
<code>main.cpp</code>	Owens all three objects above; the only file that interprets what a topic or payload <i>means</i> . Reads the sensor on a timer, reads live motor state for status publishing, applies incoming commands, and evaluates the disconnect fail-safe every loop iteration.

Non-blocking main loop

`loop()` contains no blocking `delay()` at all. WiFi and broker reconnection, the sensor read interval, and the motor status publish interval are each evaluated every pass against independent `millis()`-based timestamps, so the disconnect fail-safe check is re-evaluated on essentially every iteration regardless of what else is due.

6. Broker Setup and Network Debugging

This section documents the most time-consuming and most instructive part of the project – getting the ESP32 and a self-hosted broker to actually see each other on the LAN – since none of it was a Mosquitto configuration problem in the end.

Confirmed default behavior

Running Mosquitto 2.x with no config file at all produces a broker that only accepts local connections – confirmed directly from the broker's own startup log, not assumed from documentation. This ruled out “the default install should just work” as an explanation and pointed straight at needing an explicit `listener` directive.

```
1 listener 1883 0.0.0.0
2 allow_anonymous false
3 password_file passwd.txt
```

Listing 1: mosquitto.conf – the fix

0.0.0.0 binds the listener to every network interface on the PC, not just loopback, which is what actually lets the ESP32 reach it.

Windows network profiles are tracked per SSID

The single most important debugging finding of this project: Windows does not remember “this PC’s firewall is configured correctly” as a global fact. It tracks the Public/Private security classification of every network *independently, per SSID*. An inbound firewall rule scoped to the Private profile only actually applies while the currently active network is itself classified Private – and that classification resets the moment the PC associates with a *different* SSID, even if the new network is, physically, the same LAN. This was discovered the hard way twice in this project: once switching from a phone hotspot back to the home WiFi, and again when moving between the main router and the WiFi extender node. Each time, the previously-working setup silently stopped working, and each time the fix was the same – re-open Windows Security, confirm the active network’s profile, and set it back to Private if needed.

A second, unrelated hardware issue: extender client isolation

Separately from the firewall/profile issue, the WiFi extender node (CYBER_EXT) itself was found to block direct device-to-device LAN traffic between clients connected to it. This was confirmed to be independent of Windows Firewall by testing with the firewall fully and explicitly disabled – the connection still failed, which rules out any Windows-side explanation and points at AP-level client isolation in the extender’s own firmware. The working, verified topology places the broker-hosting PC on the *main router* directly, with the ESP32 connected through the extender; all subsequent testing in this project was done against that configuration.

Debugging method: layer isolation before adding complexity

Rather than debugging WiFi, firewall, and MQTT all at once, the network path was verified in the order it was actually built:

1. A minimal ESP32 sketch using only `WiFi.h` and a raw `WiFiClient::connect()` call – no MQTT library at all – was used to confirm plain TCP reachability first.
2. Success was confirmed by watching for matching log lines on *both* sides at once: `TCP connect OK` on the ESP32’s Serial monitor, and `New connection from...` in Mosquitto’s own verbose terminal output.
3. Only once that raw TCP path was verified working was `PubSubClient` introduced, isolating any further failures to the MQTT/application layer rather than the network layer underneath it.

This same technique paid off again later, during command testing (Section): watching Mosquitto’s own verbose log in real time made it possible to tell, immediately, whether a connection failure was a network-layer problem (`rc=-2`, TCP-level connect failure) or an authentication-layer problem (`rc=5`, credentials rejected), which are otherwise easy to conflate from the ESP32 side alone.

```

1 Opening ipv4 listen socket on port 1883.
2 mosquitto version 2.1.2 running
3 New connection from 192.168.1.106:59307 on port 1883.

```

Listing 2: Mosquitto log – raw TCP verification, no MQTT yet

Firewall rule actually applied

Setting	Value
Rule direction	Inbound
Protocol	TCP
Port	1883
Profile scope	Private only
Additional requirement	Active network profile itself set to Private (re-checked per SSID)

7. MQTT Communication Protocol

Broker connection and reconnection

MqttInterlockClient layers two independent, non-blocking reconnection routines, both driven from loop(): one for WiFi, one for the broker session. Broker reconnect attempts are rate-limited by MQTT_RECONNECT_MS so an outage does not flood reconnect attempts.

```

1 void MqttInterlockClient::loop() {
2     if (WiFi.status() != WL_CONNECTED) {
3         connectWiFi();
4     }
5     if (!_mqttClient.connected()) {
6         connectBroker();
7     }
8     _mqttClient.loop();
9 }

```

Listing 3: MqttInterlockClient.cpp – loop()

The broker's address and port live in exactly one place:

```

1 #define MQTT_BROKER      "192.168.1.33"
2 #define MQTT_PORT        1883
3 #define MQTT_USERNAME    ""
4 #define MQTT_PASSWORD    ""

```

Listing 4: config.h – broker identity (excerpt)

A unique client ID is built at connect time from a fixed prefix and the chip's efuse MAC, so re-flashing or running multiple test clients never collides on client ID:

```

1 String clientId = String(MQTT_CLIENT_ID_PREFIX) + String((uint32_t)ESP.getEfuseMac(),
  HEX);
2
3 bool connected;
4 if (strlen(MQTT_USERNAME) > 0) {
5     connected = _mqttClient.connect(clientId.c_str(), MQTT_USERNAME, MQTT_PASSWORD);
6 } else {
7     connected = _mqttClient.connect(clientId.c_str());
8 }

```

Listing 5: MqttInterlockClient.cpp – connectBroker() (excerpt)

Topic structure

Unlike the earlier project's two hardcoded, cross-wired topics, this task uses a structured, hierarchical namespace with six distinct topics:

Topic	Direction	Payload
sensor/bme280/temperature	ESP32 → broker	Float, °C
sensor/bme280/humidity	ESP32 → broker	Float, %RH
sensor/bme280/pressure	ESP32 → broker	Float, hPa
motor/status/state	ESP32 → broker	"1" running / "0" stopped
motor/status/direction	ESP32 → broker	"forward" / "reverse"
motor/status/speed	ESP32 → broker	Integer 0–100
motor/command/state	broker → ESP32	"1" run / "0" stop
motor/command/direction	broker → ESP32	"forward" / "reverse"
motor/command/speed	broker → ESP32	Integer, clamped 0–100

Table 2: Full topic namespace, structured under `sensor/` and `motor/`.

All payloads are raw ASCII values, not JSON – simple enough to type directly into MQTT Explorer or `mosquitto_pub` by hand during manual command testing.

A thin, generic transport: routing without interpreting

`MqttInterlockClient` does not know what a motor or a BME280 is. It only moves topic and payload strings; `main.cpp` owns all interpretation. The one open design question carried over from planning – whether a generic `subscribe()` could stay fully generic while still routing an incoming message to the correct one of three command slots – was resolved by tagging each subscription with a `CommandSlot` enum at call time:

```

1 enum class CommandSlot { STATE, DIRECTION, SPEED };
2
3 void subscribe(const char* topic, CommandSlot slot);

```

Listing 6: MqttInterlockClient.h – slot-tagged subscription

Matching an incoming topic string to a slot is routing, not payload interpretation, so this preserves the thin-wrapper intent while still letting the class deliver each message to the right one-shot inbox.


```

1 for (uint8_t i = 0; i < _routeCount; i++) {
2     if (_routes[i].topic == topic) {
3         switch (_routes[i].slot) {
4             case CommandSlot::STATE:      _stateInbox.value = value;      _stateInbox.hasNew =
                true;      break;
5             case CommandSlot::DIRECTION:  _directionInbox.value = value; _directionInbox.
                hasNew = true; break;
6             case CommandSlot::SPEED:      _speedInbox.value = value;      _speedInbox.hasNew =
                true;      break;
7         }
8         return;
9     }
10 }

```

Listing 7: MqttInterlockClient.cpp – handleMessage() dispatch (excerpt)

Each of the three inboxes auto-clears its “new” flag the moment `main.cpp` reads it – deliberately different from `BME280Sensor::isDataValid()`’s pure, side-effect-free read, because a command is a one-time instruction to consume, not ongoing state to keep re-checking.

8. BME280 Sensor Design

Timing ownership

`readNow()` is only ever called from a `millis()`-gated timer inside `main.cpp`’s `loop()` – the class itself has no internal timing logic, matching the project’s existing convention that peripheral timing is owned by the code that coordinates all peripherals together, not by any one sensor class.

Fail-safe caching on bad reads

```

1 if (isnan(t) || isnan(h) || isnan(p)) {
2     _lastReadSuccess = false;
3     return;
4 }
5 _temperature = t;
6 _humidity     = h;
7 _pressure     = p;
8 _lastReadSuccess = true;
9 _lastReadMs = millis();

```

Listing 8: BME280Sensor.cpp – readNow()

A failed read leaves the previously cached values untouched rather than overwriting them with `NaN` garbage – a deliberate fail-safe choice, so that a single noisy I2C transaction cannot corrupt the last known-good telemetry that continues to be published.

Single combined validity check

```

1 bool BME280Sensor::isDataValid() const {
2     bool freshEnough = (millis() - _lastReadMs) < SENSOR_STALE_MS;
3     return _sensorInitSuccess && _lastReadSuccess && freshEnough;

```

```
4 }
```

Listing 9: BME280Sensor.cpp – isValid()

Combining init success, last-read success, and a staleness window into one boolean means `main.cpp` never has to reason about three separate flags before deciding whether a reading is safe to publish.

9. Motor Control and Live Status Design

`MotorController` keeps exactly three pieces of state – `_running`, `_forward`, `_speedPercent` – and recomputes both direction pins and PWM duty from scratch in `applyOutputs()` any time one of them changes, rather than tracking derived output state separately.

```
1 void MotorController::applyOutputs() {
2     digitalWrite(MOTOR_PIN_IN1, _forward ? HIGH : LOW);
3     digitalWrite(MOTOR_PIN_IN2, _forward ? LOW : HIGH);
4
5     uint32_t maxDuty = (1 << MOTOR_PWM_RESOLUTION_BITS) - 1;
6     uint32_t duty = _running ? (maxDuty * _speedPercent) / 100 : 0;
7     ledcWrite(MOTOR_PWM_CHANNEL, duty);
8 }
```

Listing 10: MotorController.cpp – applyOutputs()

When stopped, PWM duty is forced to zero regardless of the stored speed setpoint, so the setpoint itself survives a stop/start cycle instead of being lost.

Speed clamping

```
1 if (percent < 0) { percent = 0; }
2 else if (percent > 100) { percent = 100; }
3 _speedPercent = (uint8_t)percent;
```

Listing 11: MotorController.cpp – setSpeedPercent()

Clamping lives at the actuator boundary itself, not in the MQTT command parser, so the motor is protected from an invalid setpoint no matter what ends up calling `setSpeedPercent()`.

Status reflects live state, never a cached command

```
1 mqttClient.publish(TOPIC_MOTOR_STATUS_STATE, motor.isRunning() ? "1" : "0");
2 mqttClient.publish(TOPIC_MOTOR_STATUS_DIRECTION, motor.isForward() ? "forward" : "
   reverse");
```

Listing 12: main.cpp – publishMotorStatus()

Every status publish reads `MotorController`’s live getters at publish time – never a remembered “last command sent” value. This is what directly satisfies the task requirement that status must reflect actual behavior, and it is also what makes the reconnect behavior below correct without any extra bookkeeping.

10. Fail-Safe: Disconnect and Reconnect Behavior

```

1 if (!mqttClient.isConnected()) {
2     motor.stop();
3 }

```

Listing 13: main.cpp – loop() fail-safe check

This check runs on every loop iteration, independent of any MQTT traffic – deliberately so, since publishing a “stop” command while already disconnected is not possible anyway. The motor stops locally the moment the connection is observed down, not upon receiving any particular message.

Immediate republish on reconnect, no special bookkeeping

Because status publishing always reads live state (Section 9), the requirement to publish current state once immediately upon reconnecting – rather than waiting for the next scheduled interval – falls out of a single extra call rather than any new tracking logic:

```

1 if (connected) {
2     resubscribeAll();
3     _reconnectFlag = true;
4 }

```

Listing 14: MqttInterlockClient.cpp – one-shot reconnect flag

```

1 if (mqttClient.consumeReconnectFlag()) {
2     publishMotorStatus();
3 }

```

Listing 15: main.cpp – consuming the flag

11. Manual Command Testing Methodology

With no MQTT Explorer GUI required, the full command surface was exercised from the terminal using `mosquitto_sub` (with the `#` multi-level wildcard, watching every topic at once) in one window and `mosquitto_pub` in another:

```

1 mosquitto_sub -h 192.168.1.33 -p 1883 -u <user> -P <pass> -t "sensor/#" -t "motor/#" -v

```

Listing 16: Subscribing to everything at once

```

1 mosquitto_pub -h 192.168.1.33 -p 1883 -u <user> -P <pass> -t "motor/command/state" -m "1"
2 mosquitto_pub -h 192.168.1.33 -p 1883 -u <user> -P <pass> -t "motor/command/direction" -m "reverse"
3 mosquitto_pub -h 192.168.1.33 -p 1883 -u <user> -P <pass> -t "motor/command/speed" -m "150"

```

Listing 17: Exercising each command topic

The last example intentionally sends an out-of-range value (150) to verify the clamp in `MotorController::setSpeed`. The `motor/status/speed` was confirmed to report 100, never the raw out-of-range input.

Repeated commands, reduced typing

To avoid retyping host/credential flags on every single test command, a small pair of PowerShell functions were added to the developer's profile, wrapping `mosquitto_pub/mosquitto_sub` with the broker address and credentials pre-filled:

```
1 mqp motor/command/state 1
2 mqs motor/#
```

Listing 18: PowerShell profile shortcuts

12. Status Verification without Dedicated LEDs

Unlike the prior interlock project, this build has no dedicated status LEDs – state is observed entirely through Serial output and the live MQTT topics themselves (via `mosquitto_sub`/MQTT Explorer), which is sufficient given that a broker and a terminal are already part of the required workflow for this task. This trade-off is revisited in Section 14.

13. Results & Verification

Broker verbose log

```
1786261162: Sending PUBLISH to auto-E3F11F28-10D3-3863-2D59-8AEFF6B88738 (d0, q0, r0, m0, 'sensor/bme280/humidity', ...
(5 bytes))
1786261162: Received PUBLISH from esp32-task27-a3077000 (d0, q0, r0, m0, 'sensor/bme280/pressure', ... (6 bytes))
1786261162: Sending PUBLISH to auto-E3F11F28-10D3-3863-2D59-8AEFF6B88738 (d0, q0, r0, m0, 'sensor/bme280/pressure', ...
(6 bytes))
1786261162: Received PUBLISH from esp32-task27-a3077000 (d0, q0, r0, m0, 'motor/status/state', ... (1 bytes))
1786261162: Sending PUBLISH to auto-E3F11F28-10D3-3863-2D59-8AEFF6B88738 (d0, q0, r0, m0, 'motor/status/state', ... (1 b
ytes))
1786261162: Received PUBLISH from esp32-task27-a3077000 (d0, q0, r0, m0, 'motor/status/direction', ... (7 bytes))
1786261162: Sending PUBLISH to auto-E3F11F28-10D3-3863-2D59-8AEFF6B88738 (d0, q0, r0, m0, 'motor/status/direction', ...
(7 bytes))
1786261162: Received PUBLISH from esp32-task27-a3077000 (d0, q0, r0, m0, 'motor/status/speed', ... (3 bytes))
1786261162: Sending PUBLISH to auto-E3F11F28-10D3-3863-2D59-8AEFF6B88738 (d0, q0, r0, m0, 'motor/status/speed', ... (3 b
ytes))
1786261163: Received PUBLISH from esp32-task27-a3077000 (d0, q0, r0, m0, 'motor/status/state', ... (1 bytes))
1786261163: Sending PUBLISH to auto-E3F11F28-10D3-3863-2D59-8AEFF6B88738 (d0, q0, r0, m0, 'motor/status/state', ... (1 b
ytes))
1786261163: Received PUBLISH from esp32-task27-a3077000 (d0, q0, r0, m0, 'motor/status/direction', ... (7 bytes))
1786261163: Sending PUBLISH to auto-E3F11F28-10D3-3863-2D59-8AEFF6B88738 (d0, q0, r0, m0, 'motor/status/direction', ...
(7 bytes))
1786261163: Received PUBLISH from esp32-task27-a3077000 (d0, q0, r0, m0, 'motor/status/speed', ... (3 bytes))
1786261163: Sending PUBLISH to auto-E3F11F28-10D3-3863-2D59-8AEFF6B88738 (d0, q0, r0, m0, 'motor/status/speed', ... (3 b
ytes))
1786261164: Received PUBLISH from esp32-task27-a3077000 (d0, q0, r0, m0, 'motor/status/state', ... (1 bytes))
1786261164: Sending PUBLISH to auto-E3F11F28-10D3-3863-2D59-8AEFF6B88738 (d0, q0, r0, m0, 'motor/status/state', ... (1 b
ytes))
1786261164: Received PUBLISH from esp32-task27-a3077000 (d0, q0, r0, m0, 'motor/status/direction', ... (7 bytes))
1786261164: Sending PUBLISH to auto-E3F11F28-10D3-3863-2D59-8AEFF6B88738 (d0, q0, r0, m0, 'motor/status/direction', ...
(7 bytes))
1786261164: Received PUBLISH from esp32-task27-a3077000 (d0, q0, r0, m0, 'motor/status/speed', ... (3 bytes))
```

Figure 3: Mosquitto verbose output showing the ESP32 connecting and publishing

Live telemetry and status in MQTT Explorer / terminal

```
motor/status/state 1
motor/status/direction reverse
motor/status/speed 100
sensor/bme280/temperature 23.49
sensor/bme280/humidity 40.17
sensor/bme280/pressure 892.95
motor/status/state 1
motor/status/direction reverse
motor/status/speed 100
```

Figure 4: Sensor telemetry and motor status topics updating on their respective intervals

Command-driven motor control

```
PS C:\Users\User\Documents\PlatformIO\Projects\project 7> . $PROFILE
PS C:\Users\User\Documents\PlatformIO\Projects\project 7> mqp motor/command/state 0
PS C:\Users\User\Documents\PlatformIO\Projects\project 7> mqp motor/command/state 1
>>
PS C:\Users\User\Documents\PlatformIO\Projects\project 7> mqp motor/command/direction reverse
>>
PS C:\Users\User\Documents\PlatformIO\Projects\project 7> mqp motor/command/speed 75
>>
PS C:\Users\User\Documents\PlatformIO\Projects\project 7> mqs motor/#
>>
motor/status/direction reverse
motor/status/speed 75
motor/status/state 1
motor/status/direction reverse
motor/status/speed 75
```

Figure 5: A published command on motor/command/* immediately reflected in motor/status/*

Fail-safe verification

```
Initialized BME280
Connecting to WiFi...
WiFi connected, IP: 192.168.1.106
Connecting to MQTT broker 192.168.1.33 ... connected.
Subscribed to: motor/command/state
Subscribed to: motor/command/direction
Subscribed to: motor/command/speed
[recv] topic=motor/command/speed len=3 payload=100
Connecting to MQTT broker 192.168.1.33 ... failed, rc=-2
Connecting to MQTT broker 192.168.1.33 ... failed, rc=-2
```

Figure 6: Motor stopping locally on disconnect, and status republishing immediately on reconnect

14. Design Notes – Efficient Choices

1. **Reused only what was still true.** The WiFi/broker reconnect pattern and the `MqttInterlockClient` name carried over from the interlock project; everything tied to the two-board partner concept was deliberately dropped rather than adapted, since keeping unused interlock machinery around would have made the thin-wrapper design harder to reason about.
2. **Structured topic namespace from the start.** Six topics under `sensor/` and `motor/`, rather than a flat list, keeps the namespace legible as more sensors or actuators are added later.
3. **Live-state status, not last-command status.** Reading `MotorController`'s getters at publish time, rather than caching the last applied command, is what makes both the core status requirement and the reconnect-republish behavior correct simultaneously, with no duplicated logic between them.
4. **Actuator-boundary clamping.** Speed validation lives inside `MotorController` itself, not in the MQTT layer, so the motor is protected regardless of what future code paths end up calling `setSpeedPercent()`.
5. **Routing without interpreting.** Tagging each `subscribe()` call with a `CommandSlot` lets a fully generic MQTT class still deliver messages to the correct one-shot inbox, without the class ever needing to understand what a state, direction, or speed value actually means.
6. **Layered network debugging.** Verifying raw TCP reachability before introducing MQTT at all cleanly separated two completely different classes of failure – network/firewall vs. application-layer – and the same broker-log-watching habit later separated a TCP-level failure (`rc=-2`) from an authentication-level one (`rc=5`) during command testing.
7. **Non-blocking loop.** No blocking `delay()` anywhere in `loop()`; sensor reads, status publishes, and both reconnect routines are independently timer-gated against `millis()`.

15. Known Limitations & Considerations

- **Unencrypted transport.** The connection uses plain MQTT_PORT (1883), not TLS, so payloads and credentials travel in clear text on the local network.
- **No QoS or delivery guarantee.** `PubSubClient` defaults to QoS 0 for both publish and subscribe; an individual dropped message is not retried. This is tolerated for sensor data (published continuously) but is a real gap for one-shot motor commands, which are not resent if lost in transit.
- **No Last Will and Testament.** An ungraceful ESP32 disconnect (power loss, crash) is only inferred by the absence of further publishes – there is no broker-side LWT message marking the device as offline the instant the TCP session drops.
- **Network topology dependency.** The verified working setup requires the broker-hosting PC to sit on the main router directly, with the ESP32 on the extender – a direct consequence of the extender's client-isolation behavior documented in Section , not something the firmware can work around on its own.
- **No dedicated status LEDs.** Unlike the earlier four-LED interlock build, this project relies entirely on Serial output and live MQTT topics for observability, which is adequate for bench testing but offers no at-a-glance state indication without a terminal or MQTT client open.

- **Credentials in config.h.** WiFi and MQTT credentials are plain `#define` constants committed alongside the rest of the source.

16. Bill of Materials

Item	Qty	Notes
ESP32 DevKit development board	1	Runs the full firmware described in this report
BME280 environmental sensor module	1	I2C, address 0x76, default <code>Wire</code> pins
L298N (or equivalent) H-bridge motor driver	1	Driven via IN1/IN2/ENA
DC motor	1	Bidirectional, speed-controlled via PWM
Breadboard + jumper wires	1 set	–
USB cable	1	Programming and power
PC running Mosquitto 2.1.2	1	Broker host, same LAN as the ESP32
WiFi network (main router + extender)	–	See Section for topology constraints

17. Future Improvements

- **TLS transport** on the broker's secure port, so payloads and credentials no longer travel in clear text.
- **Last Will and Testament** on the broker connection, so an ungraceful ESP32 disconnect is signaled immediately on a dedicated topic rather than only inferred from silence.
- **QoS 1** on the three command topics specifically, so a motor command is broker-acknowledged rather than best-effort, while sensor/status topics can remain QoS 0.
- **Retained messages** on the `motor/status/*` topics, so a newly-connecting MQTT client immediately sees the last known motor state without waiting for the next publish.
- **Broker-side ACLs** scoped per topic prefix, once multiple devices or students share the same broker instance.
- **Resolve the extender client-isolation constraint** directly – either by replacing the extender, enabling any AP-isolation override it might expose, or moving the broker onto the same access point as the ESP32 – to remove the current PC-on-router / ESP32-on-extender topology requirement.